

UML. Типы диаграмм в UML.

Диаграмма состояний (конечного автомата)

Назначение диаграммы состояний



- **Диаграммы состояний (State machine)** предназначены для описания поведения элемента модели в течение его жизненного цикла:
 - применяются для того, чтобы объяснить, каким образом работают сложные объекты;
 - полезна при моделировании жизненного цикла объекта;
 - служат для моделирования динамических аспектов системы.

Основные понятия

- **Диаграмма состояний** является ориентированным **графом**, который представляет некоторый конечный автомат.
- **Конечный автомат** (*state machine*) – некоторый формализм для моделирования поведения отдельных элементов модели или системы в целом.
- **Поведение** (*behavior*) является спецификацией того, как экземпляр классификатора изменяет значения отдельных характеристик в течение своего времени жизни.
- **Состояние** (*state*) – элемент модели поведения, предназначенный для представления ситуации, в ходе которой поддерживается некоторое условие инварианта.
- **Переход** (*transition*) является направленным отношением между двумя состояниями, одно из которых является вершиной *источником* (source vertex), а другое – *целевой вершиной* (target vertex).

Обязательные условия построения автомата

- Автомат не запоминает историю перемещения из состояния в состояние.
- В каждый момент времени автомат может находиться в одном и только в одном из своих состояний.
- Хотя процесс изменения состояний автомата происходит во времени, явно концепция времени не входит в формализм автомата.
- *Количество состояний* автомата должно быть обязательно *конечным*.
- Граф автомата не должен содержать изолированных состояний и переходов.
- Автомат не должен содержать конфликтующих переходов.

Основные обозначения на диаграмме состояний



ПСЕВДОСОСТОЯНИЯ (НАЧАЛЬНОЕ И КОНЕЧНОЕ СОСТОЯНИЯ)

- **Псевдосостояние** (*pseudo-state*) – вершина в конечном автомате, которая имеет форму состояния, но не обладает поведением:
 - **Начальное состояние** (start state) – разновидность псевдосостояния, обозначающее начало выполнения процесса изменения состояний конечного автомата или нахождения моделируемого объекта в составном состоянии. В этом состоянии находится объект по умолчанию в начальный момент времени.
 - **Конечное состояние** (final state) – разновидность псевдосостояния, обозначающее прекращение процесса изменения состояний конечного автомата или нахождения моделируемого объекта в составном состоянии. В этом состоянии объект будет находиться по умолчанию после завершения работы автомата в конечный момент времени.



начальное состояние



конечное состояние

ПРОСТОЕ СОСТОЯНИЕ (*simple state*)

- **Простое состояние** – состояние, которое не имеет внутренних регионов и подсостояний

Имя состояния

Ожидание
ввода
пароля

Открытие счета

Имя состояния

Список внутренних
действий в данном
состоянии

Имя состояния

- раскрывает содержательный смысл данного состояния записывается с заглавной буквы
- должно быть уникальным
- рекомендуется в качестве имени использовать **глаголы в настоящем времени** (**ожидает**) или соответствующие **причастия** (**ожидание**)
- имя может отсутствовать (**анонимное состояние**), все анонимные состояния считаются различными

- **Простое состояние может включать в себя внутреннюю деятельность**

Секция внутренней деятельности

- **entry** – эта метка специфицирует поведение, которое также называют *входным поведением*. Это поведение выполняется всякий раз, когда происходит вход в данное состояние независимо от перехода, позволившего достичь это состояние (*действие на входе*).
- **exit** – эта метка специфицирует поведение, которое также называют *выходным поведением*. Это поведение выполняется всякий раз, когда происходит выход из данного состояния независимо от перехода, который выводит из этого состояния (*действие на выходе*).
- **do** – эта метка специфицирует поведение, которое выполняется до тех пор, пока моделируемый элемент находится в данном состоянии, или до тех пор, пока не закончится выполнение деятельности, специфицированной соответствующим выражением (*до деятельность*).
- **include** – эта метка специфицирует обращение к подавтомату (указывается имя подавтомата).
- **defer** – событие, обработка которого предписывается в другом состоянии, но после того, как все операции в текущем будут завершены.

<метка действия '/' выражение действия>

Простое состояние с внутренними действиями

entry / action-expression

do / activity-expression

exit / action-expression

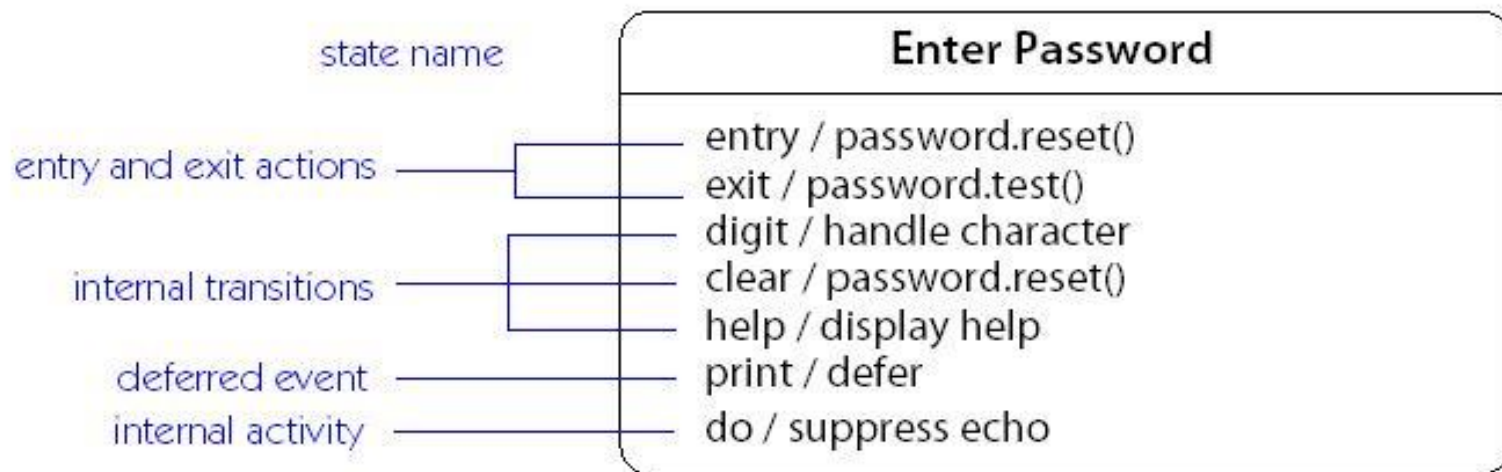
event-name / defer

include machine-name(argument_{list})_{opt}

Activity (деятельность) символизирует состояние, в котором объект находится

продолжительное количество времени.

Action (действие) выполняется **моментально.**



ПРОСТОЙ ПЕРЕХОД (*simple transition*)

- **Простой переход** (simple transition) представляет собой отношение между двумя последовательными состояниями, которое указывает на факт смены одного состояния другим.
- **Переход** осуществляется *при наступлении некоторого события*:
 - окончания выполнения деятельности (do activity),
 - получении объектом сообщения
 - приемом сигнала.

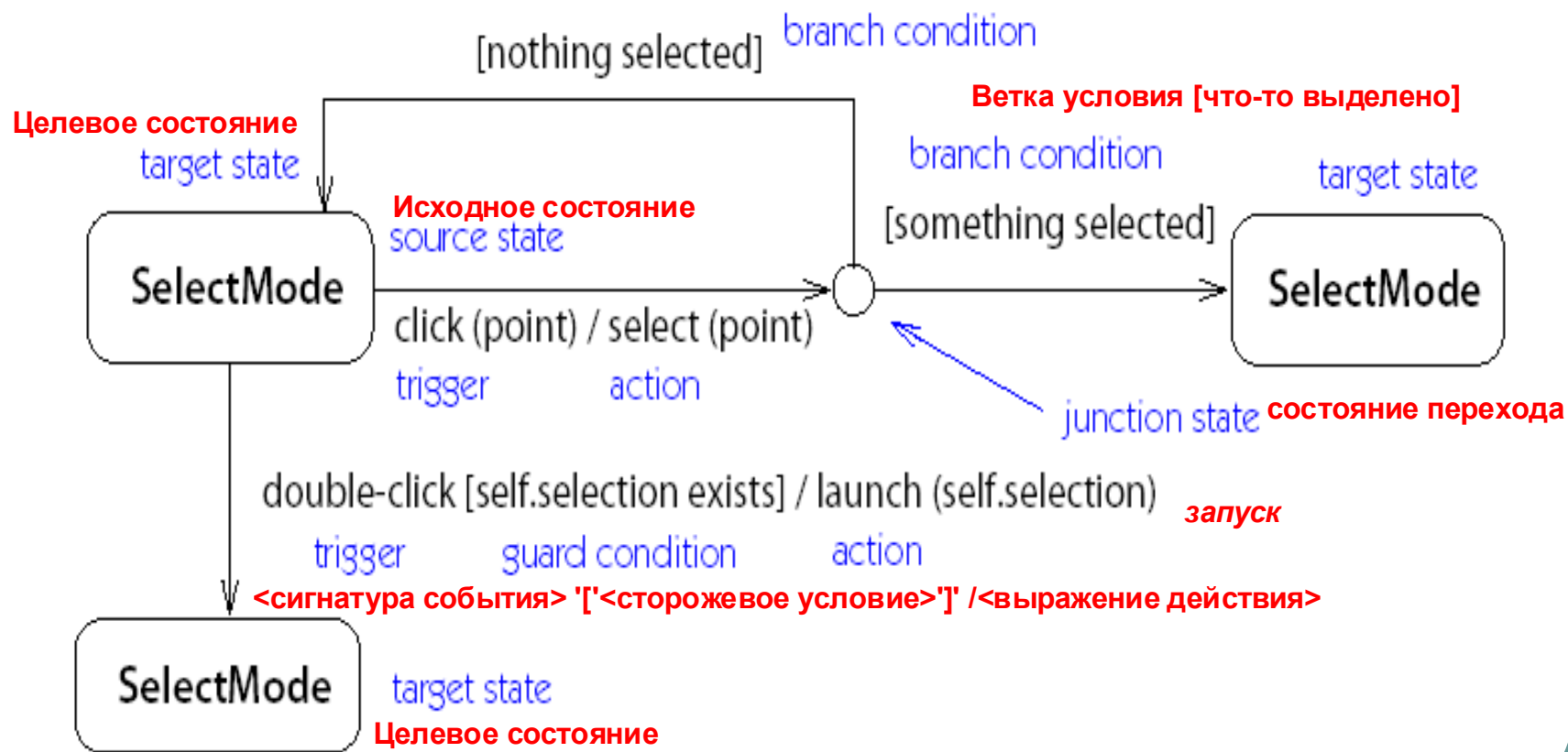
<событие> [<условие>] / <действие>
- На переходе указывается имя события.
 - <сигнатура события> '[' <сторожевое условие> ']' <выражение действия>
 - <сигнатура события> '::=<имя события>' ('<список параметров, разделенных запятыми>')



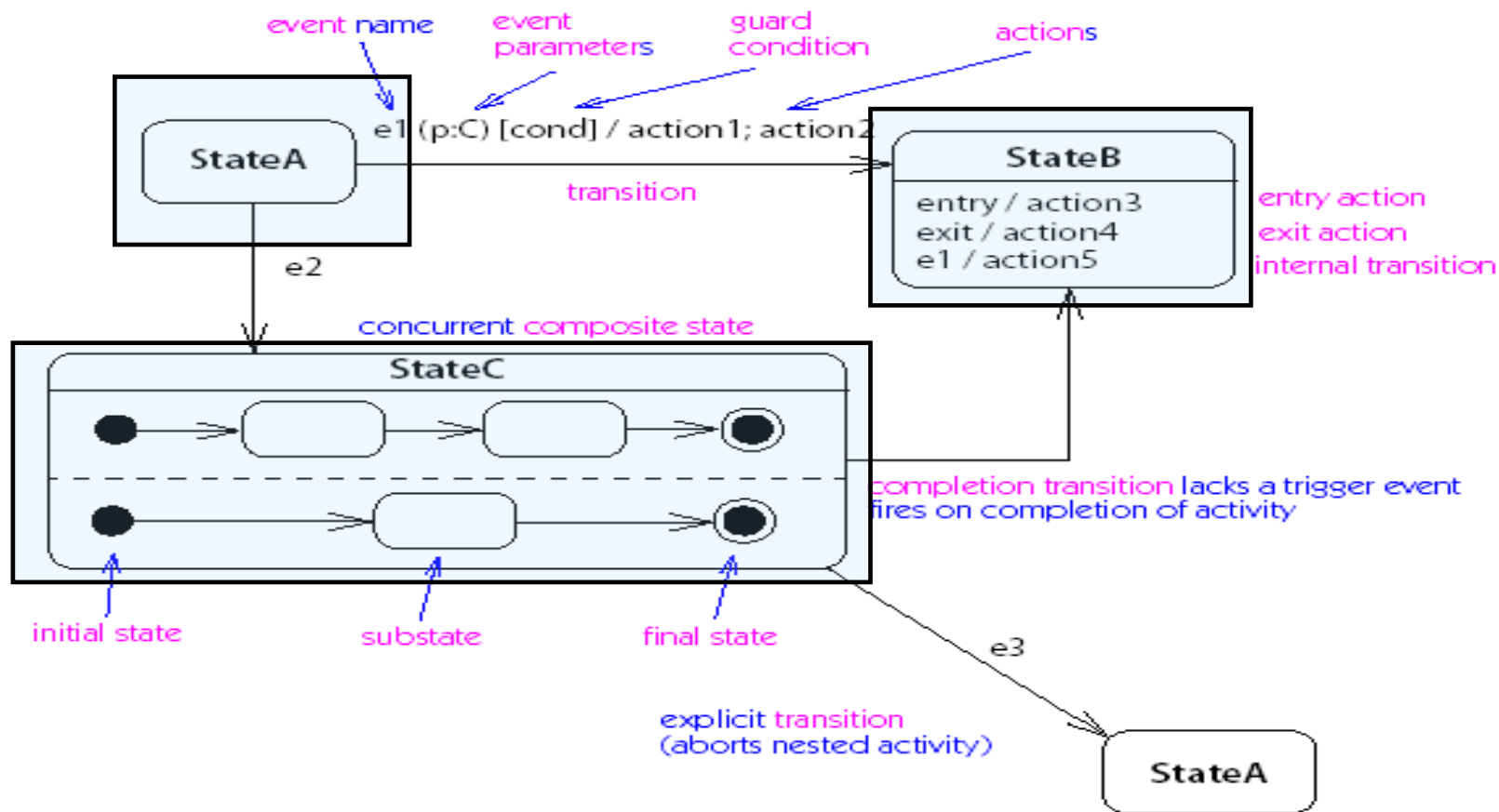
СОСТАВНОЙ ПЕРЕХОД (*compound transition*)

- **Составной переход** является производным семантическим понятием, которое представляет “семантически полный” путь, совершаемый одним или несколькими переходами.
- *Передача сигнала* является действием, которое имеет специальную графическую нотацию, описанную ранее при рассмотрении диаграмм деятельности.
- *Прием сигнала*, который также называют *приемом триггера*, является действием, которое имеет специальную графическую нотацию, описанную ранее при рассмотрении диаграмм деятельности.
- Синтаксис приема сигнала:
$$\langle \text{прием-сигнала} \rangle :: = \langle \text{триггер} \rangle [', ' \langle \text{триггер} \rangle]^* \\ ['[\langle \text{сторожевое-условие} \rangle ']]$$

СОСТАВНОЙ ПЕРЕХОД (пример)



Примеры обозначений для конечного автомата



Конфликтующие переходы

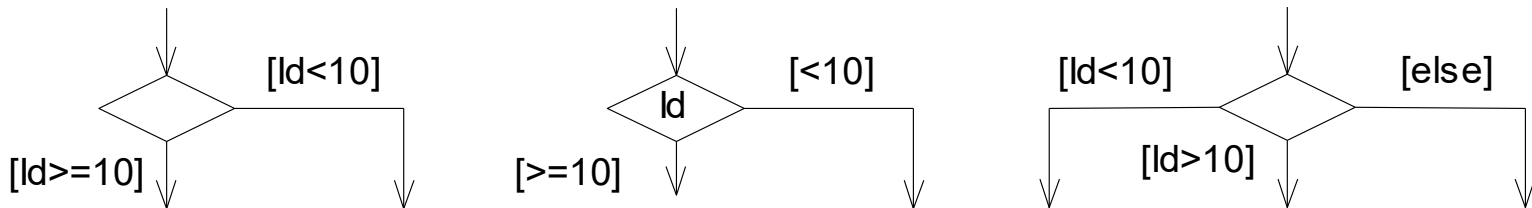
- Два и более разрешенных перехода называются **конфликтующими** (conflicting), если все они выходят из одного и того же состояния, или, более точно, пересечение множеств состояний источников не является пустым.

Пример конфликта переходов и вариант устранения конфликта

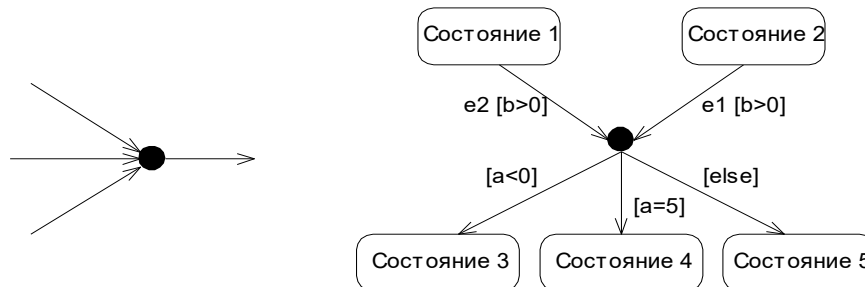


ВЫБОР (ВЕТВЛЕНИЕ) И СОЕДИНЕНИЕ

- **Псевдосостояние выбора** (*choice pseudo state*) предназначено для моделирования нескольких альтернативных ветвей при реализации поведения конечного автомата.



- **Псевдосостояние соединения** (*junction pseudo state*) является вершиной со свободной семантикой, которая используется для соединения вместе нескольких переходов.



СОСТАВНОЕ СОСТОЯНИЕ и подсостояние

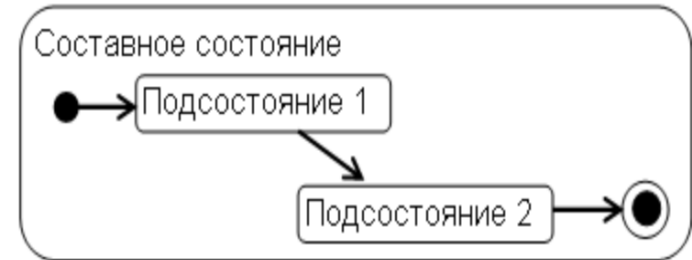
- **Составное состояние** (composite state) – такое сложное состояние, которое состоит из других вложенных в него состояний (регионов).
- позволяет учесть более тонкие логические аспекты его внутреннего поведения
 - *Регион (region)* – специальный элемент модели, который содержит состояния и переходы.
- **Составное состояние** может содержать два или более параллельных подавтомата или несколько последовательных подсостояний.
 - *любое из подсостояний, в свою очередь, может являться составным состоянием и содержать внутри себя другие вложенные подсостояния.*
 - *количество уровней вложенности составных состояний не фиксировано.*



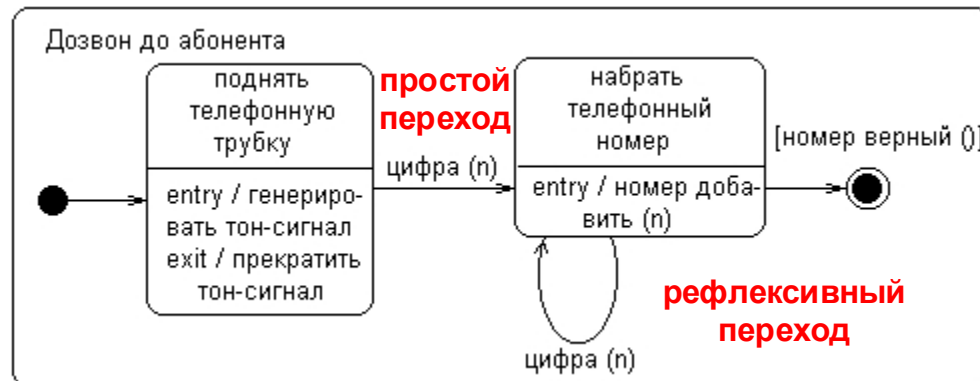
СОСТАВНОЕ СОСТОЯНИЕ

с последовательными подсостояниями

- **Последовательные подсостояния** (*sequential substates*) используются для моделирования такого поведения объекта, во время которого в каждый момент времени объект может находиться в одном и только одном из подсостояний.



Пример составного состояния с двумя вложенными последовательными подсостояниями

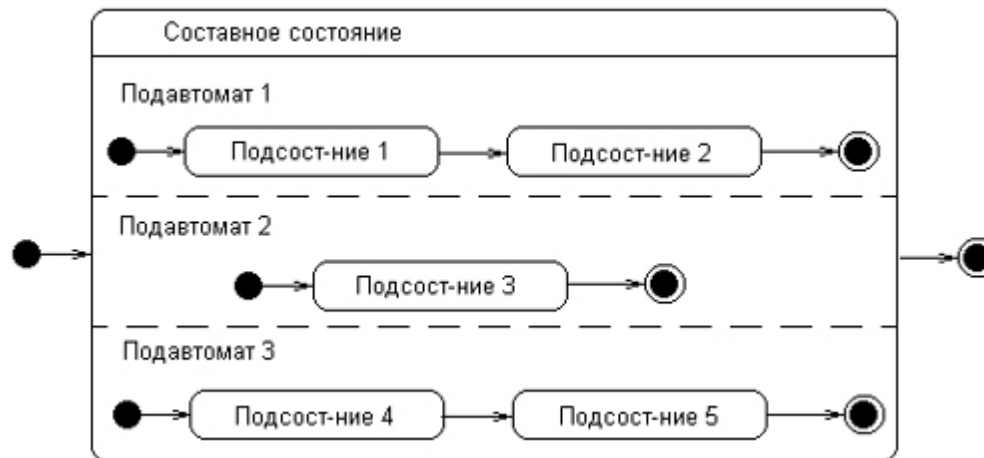


СОСТАВНОЕ СОСТОЯНИЕ

с параллельными подсостояниями

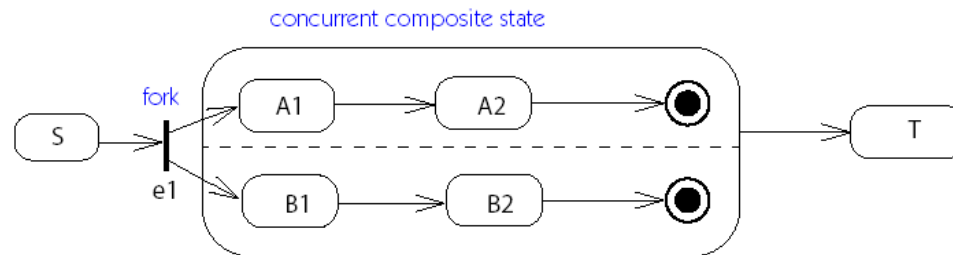
- **Параллельные подсостояния** (*concurrent substates*) позволяют специфицировать два и более подавтомата, которые могут выполняться параллельно внутри составного события.
- параллельные подсостояния могут состоять из нескольких последовательных подсостояний.

Пример составного состояния с вложенными параллельными подсостояниями



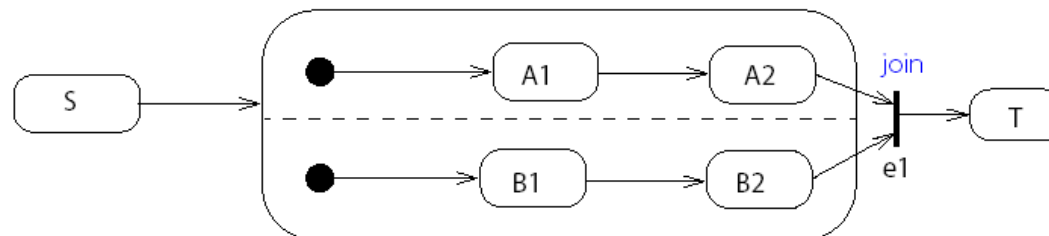
Разделение и слияние

- **Вершина разделения (fork vertex)** – псевдосостояние, предназначенное для разделения входящего перехода на два или более перехода, которые имеют в качестве своих целей вершины в ортогональных регионах композитного состояния.



When e1 occurs, A1 and B1 become active.

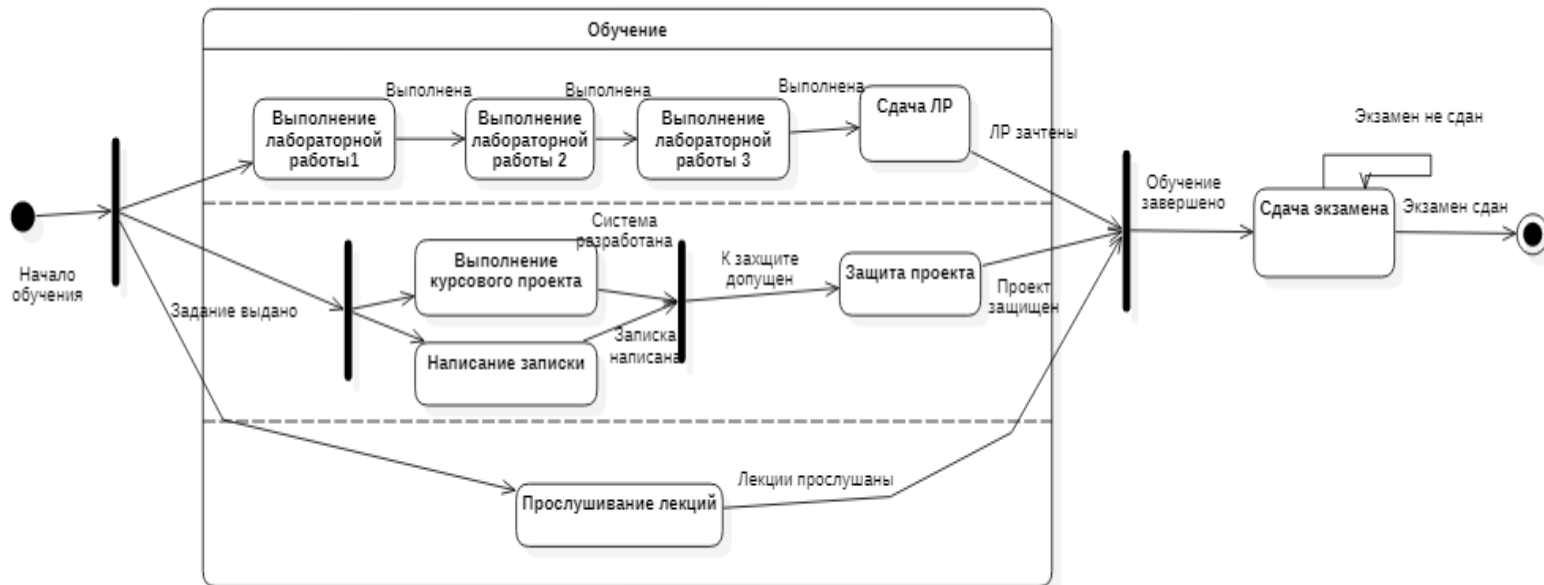
- **Вершина слияния (join vertex)** – псевдосостояние, предназначенное для соединения нескольких переходов, которые имеют в качестве своих источников вершины из различных ортогональных регионов композитного состояния.



СОСТАВНОЕ СОСТОЯНИЕ

с параллельными подсостояниями и сложными переходами

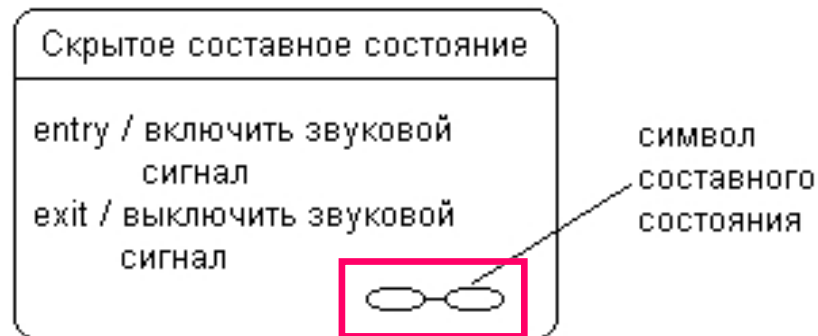
- Пример составного состояния



СОСТАВНОЕ СОСТОЯНИЕ

со скрытой внутренней структурой

- позволяет скрыть внутреннюю структуру составного состояния
- используется для больших по масштабу составных состояний



Историческое состояние

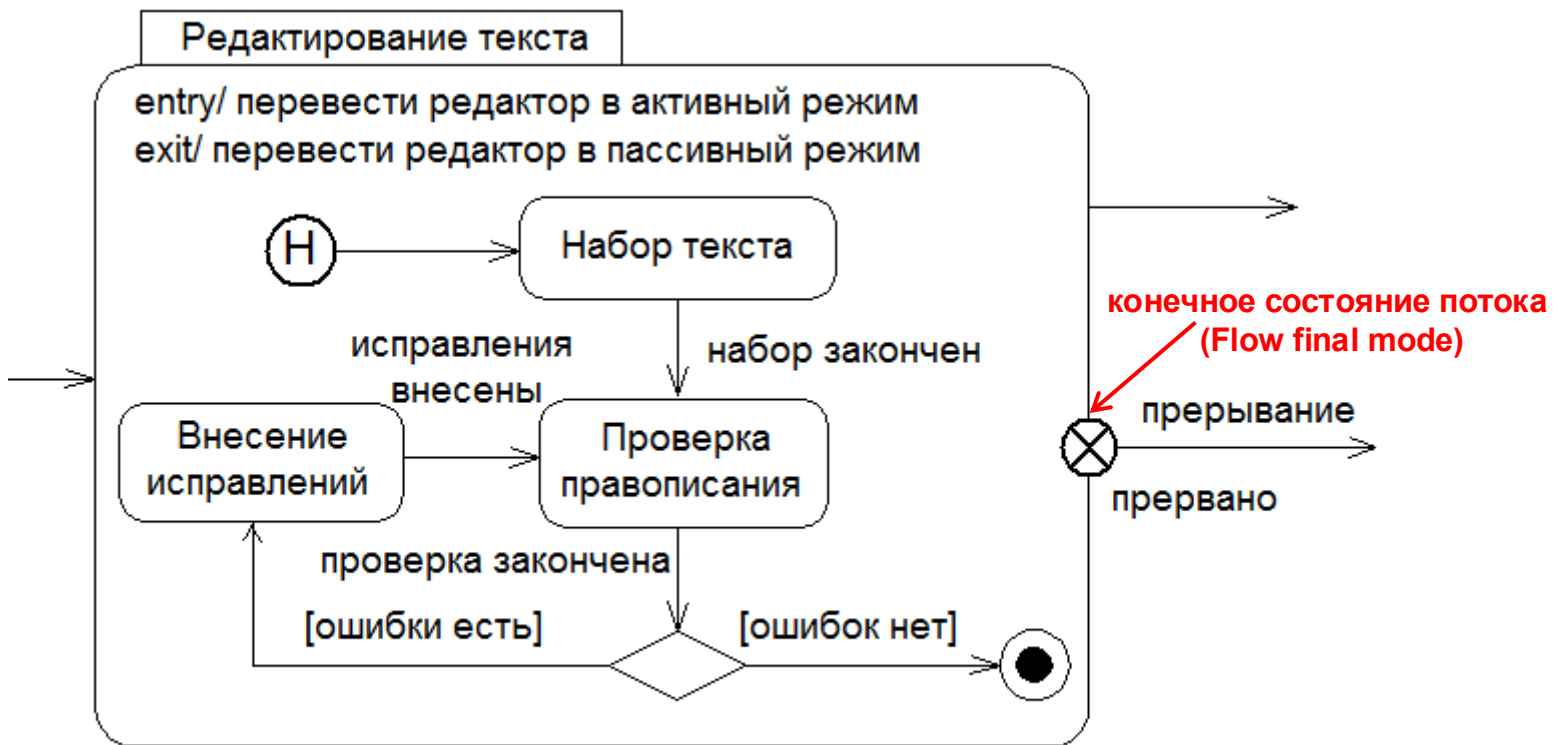
- **Историческое состояние** (*history state*) используется для запоминания того из последовательных подсостояний, которое было текущим в момент выхода из составного состояния.
- Существует две разновидности исторического состояния:
 недавнее и *давнее*.



- *недавнее историческое состояние* (shallow history state) запоминает историю только того подавтомата, к которому он относится
- *давнее историческое состояние* (deep history state) служит для запоминания всех подсостояний любого уровня вложенности для текущего подавтомата

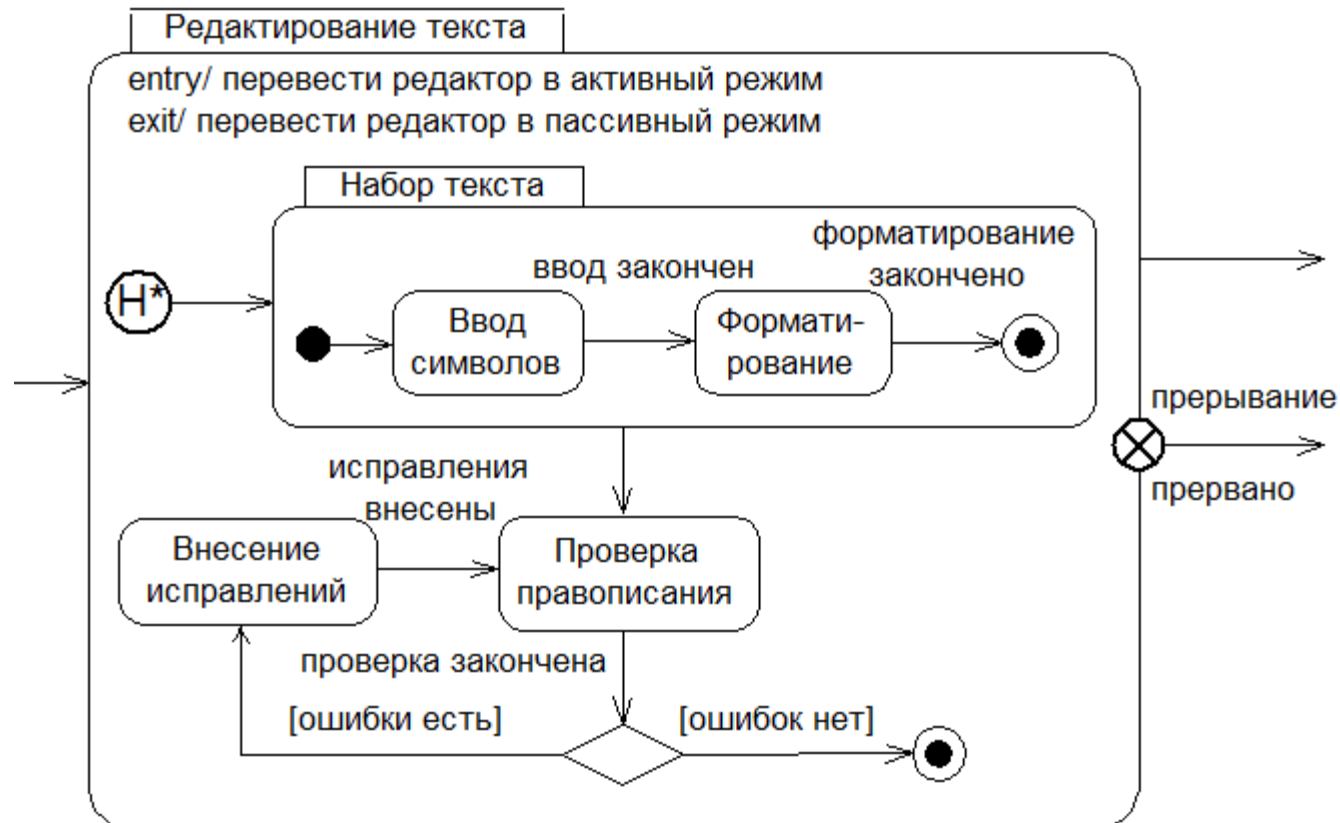
Недавнее историческое состояние

- запоминает историю только того подавтомата, к которому он относится



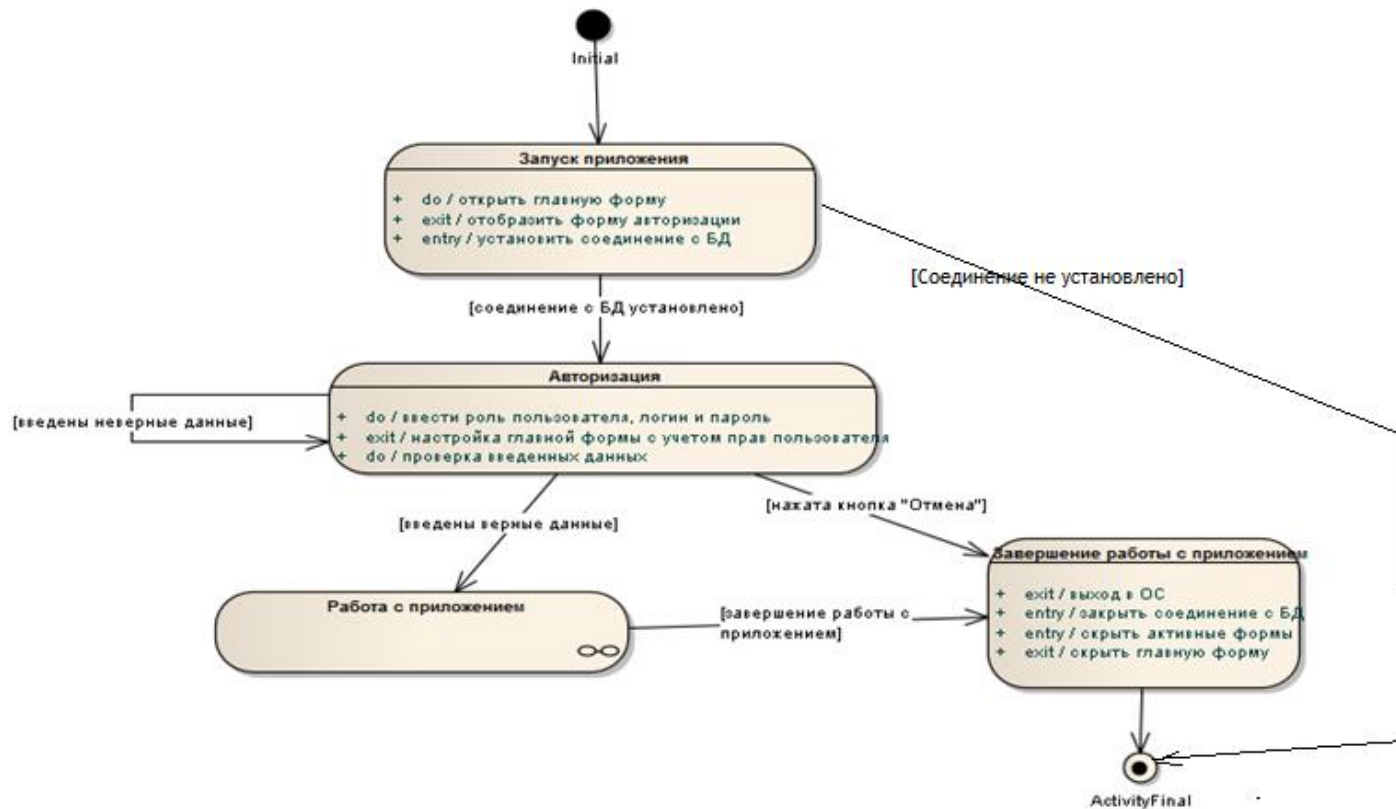
Давнее историческое состояние

- служит для запоминания всех подсостояний любого уровня вложенности для текущего подавтомата



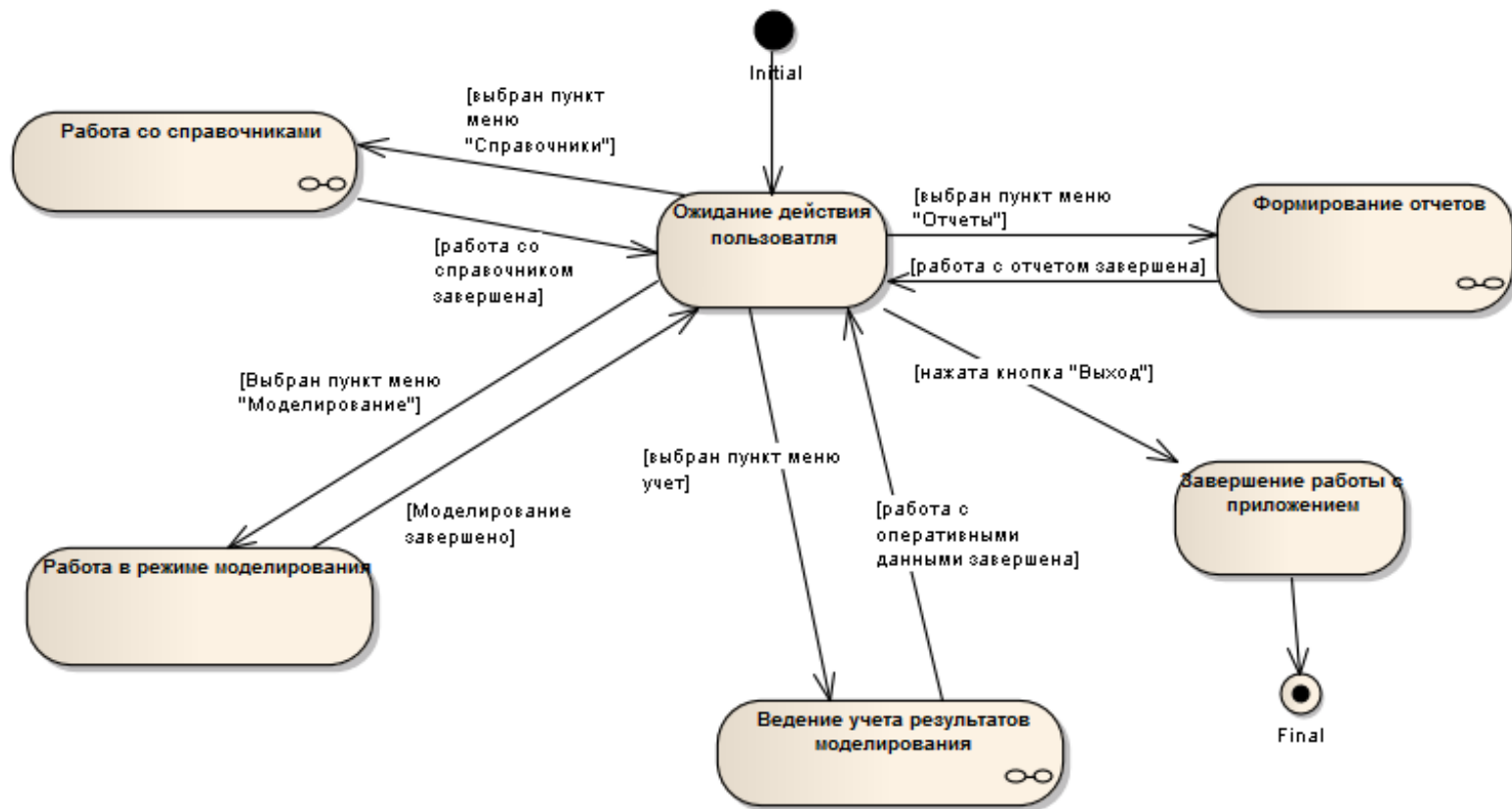
Пример диаграммы состояний

Обобщенная диаграмма состояний системы



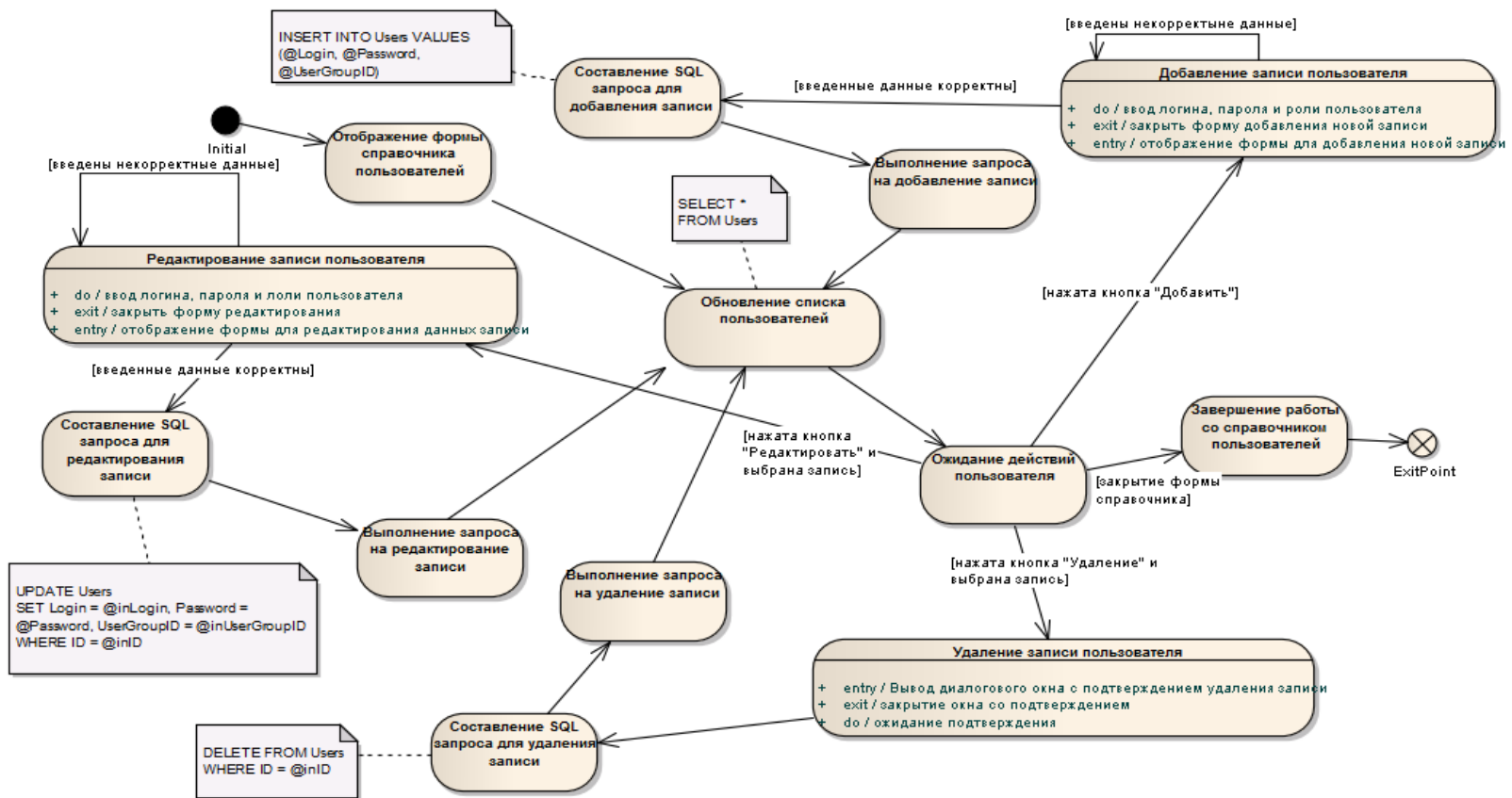
Пример диаграммы состояний

Декомпозиция состояния "Работа с приложением"



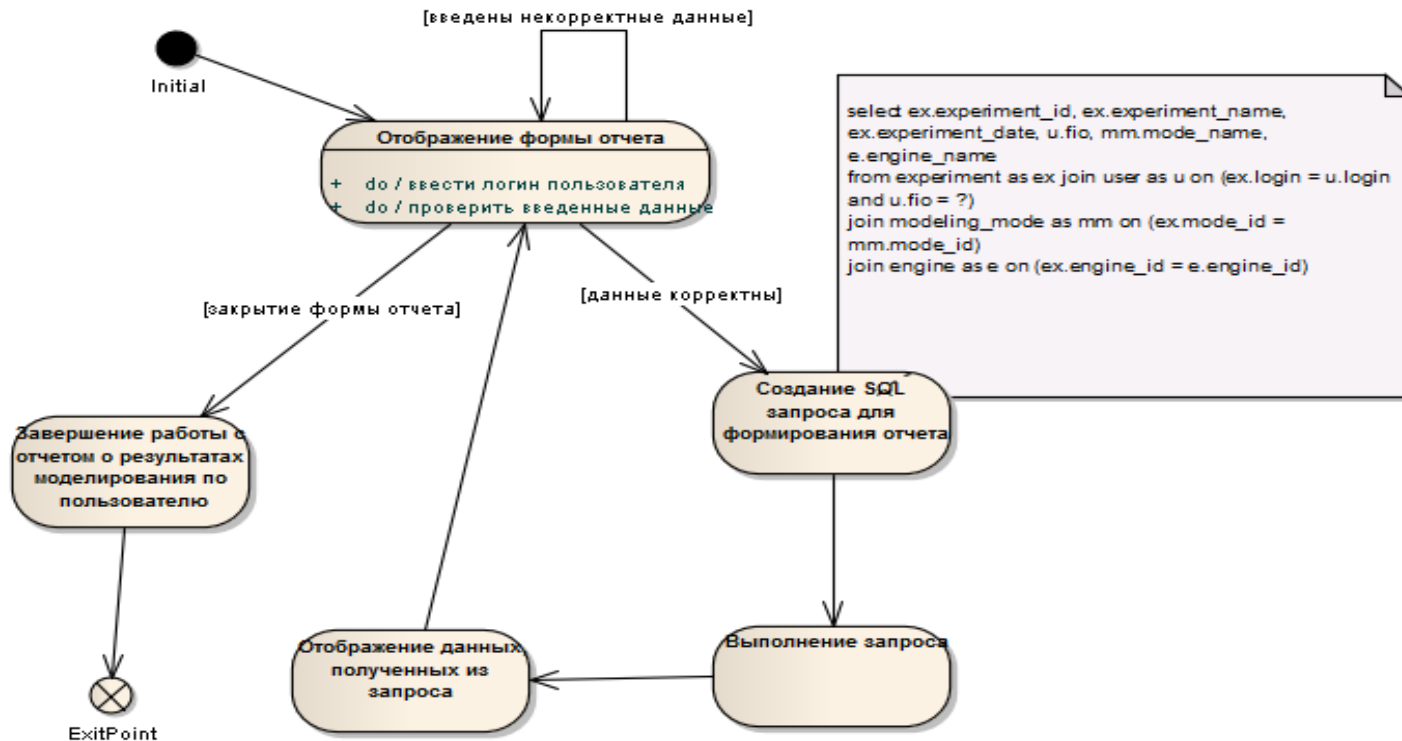
Пример диаграммы состояний

Диаграмма состояний ведения справочника пользователей



Пример диаграммы состояний

Диаграмма состояний формирования отчета



РЕКОМЕНДАЦИИ

по построению диаграммы состояний

- При выделении состояний и переходов следует помнить, что длительность срабатывания отдельных переходов должна быть существенно меньшей, чем нахождение моделируемого объекта в соответствующих состояниях.
- Все переходы должны быть явно специфицированы.
- Объект в каждый момент должен находиться только в единственном состоянии.
- Проверять отсутствие конфликтов у переходов.
- При наличии параллельности следует заменить конфликтующие переходы одним параллельным переходом типа ветвления.
- Использование исторических состояний оправдано в том случае, когда необходимо организовать обработку исключительных ситуаций (прерываний) без потери данных или выполненной работы.